

Poirot 项目简历写法报告

业务型 Agent 项目选择报告 · 源码验证版 · 简历改造路线

生成日期	2026-08-24
推荐模式	agent-only（只按 Agent 工程岗口径撰写）
用户画像	Agent 工程岗（后端实习 / Agent 工程 / 大模型应用方向），改造时间预算 2-3 周，在校生 / 早期候选人
筛选口径	本次走"当前项目简历描述"路径——项目已确定（本地源码即证据源），跳过候选池构建、短名单确认与远程拉取环节；业务型 Agent 标准（业务数据、状态流转、持久化、工具调用、评测、用户价值）逐项核验。
可信度边界	所有"负责功能 / 技术难点"均来自本地源码核验（poirot/ 目录、commit c1880f8，共 202 个 commit）；"建议简历功能点"是完成对应改造后才能写的内容，不得提前写进简历。证据统一收在"代码验证摘要"，简历条目按真实简历口吻撰写、不内嵌证据标签。

结论先行

- 项目定位：业务型 Agent——深度研究 Agent 内核，覆盖"研究"这个完整业务闭环（提出问题 → 规划检索 → 工具调用 → 记忆沉淀 → 结构化结论交付）。
- 模式解释：采用 agent-only 是因为目标岗位是 Agent 工程岗，简历口径只突出 Agent 工程机制（工具调用、长期记忆、上下文治理、多智能体编排、安全守卫、可观测性），不混入后端 CRUD 类表达。
- 为什么适合写简历：该项目的机制密度高且均可被追问——预算熔断、死循环熔断、记忆衰减、异步固化、多模型降级、沙箱路径强制都是"能讲清实现细节"的面试弹药；六项业务型 Agent 门槛全部命中：业务数据（研究结论与报告）、状态流转（线程状态 + 运行日志）、持久化（traces.md 记忆文件 + SQLite skill 存储）、工具调用（MCP / 内置工具）、评测（L3 评测框架 + 三层 skill 评测 + 2400+ 测试）、用户价值（深度调研助手）。
- 风险提示：项目机制复杂度高，面试官可能任选一条深挖；写进简历前必须保证每条都能讲清数据流、失败处理和取舍（见"落地计划"）。

项目定位

- 项目名称：Poirot（深度研究 Agent 内核）
- 定位：业务型 Agent / 新奇项目——关注"Agent 是怎么被架构出来的"，模块解耦、独立可测
- 技术栈（来自 pyproject.toml / README）：Python 3.12+、LangGraph 1.x、LangChain、SQLite、Docker、Textual (TUI)、DeepSeek (模型)
- 核心模块全景：ReAct 研究内核 (LeaderAgent + 21 个 middleware) → 上下文工程治理 → 五层长期记忆 → 多智能体编排 (specialist 委派 + subagent 自拷贝) → Docker 沙箱隔离 → MCP 工具生态 → 三层 skill 自进化 → 双端 UI + 多模型降级 + RunJournal 可观测性

已有能力（源码已验证）

#	能力	证据位置（本地源码）
1	上下文预算治理：按消息 id 去重累计 token，fraction = 当前用量 / 真实模型窗口，触发 P1 外化 / P4 压缩 / P5 熔断	poirot/backend/agents/context_engineering/strategies/default/budget.py (BudgetTrackerExecutor)
2	多模型故障降级链：仅瞬时故障（超时 / 限流 / 5xx）降级，客户端错误直接抛，记忆活跃 provider	poirot/backend/agents/config/fallback_model.py (FallbackChatModel._should_fallback)
3	ReAct 死循环熔断：近 10 条消息按（工具名, 参数哈希）去重，同调用 3 次即熔断收尾	poirot/backend/agents/middlewares/loop_detection_middleware.py (LoopDetectionMiddleware)
4	长期记忆衰减：Ebbinghaus 公式惰性计算，strength = base×(1-decay)^hours + log(1+access)×0.1 + importance×0.05	poirot/backend/agents/memory/decay_policy.py (DecayPolicy 契约)

#	能力	证据位置（本地源码）
5	记忆持久化：单文件 traces.md 作为 truth source，内存索引可重建，线程锁 + 解析容错	poirot/backend/agents/memory/strategies/default/store.py (MarkdownFileStore)
6	记忆异步固化：daemon 线程 + 队列，LLM 抽取 JSON 容错，最旧 10 条合并为语义记忆，失败仅记日志	poirot/backend/agents/memory/worker.py (MemoryWorker)
7	沙箱路径双向映射：容器路径直传 + 反向映射回宿主主机物理路径，前缀白名单校验	poirot/backend/agents/sandbox/translators/docker_path_translator.py (DockerPathTranslator)
8	沙箱写路径强制：写文件与 bash 重定向目标必须落挂载区，越界抛 SandboxPermissionError	poirot/backend/agents/sandbox/guards/docker_path_guard.py (DockerPathGuard)
9	第三方 MCP 凭据脱敏：ghp_* / sk-* / Bearer * / token=* → [REDACTED]	poirot/backend/agents/mcp/guards/credential_sanitizer.py (CredentialSanitizer)
10	L3 评测框架：programmatic (success_criteria + Wilson 95% CI) / LLM judge / 纵向配对三种适配器 + SQLite 评测记录 + CLI 命令	poirot/backend/agents/multiagent/eval/ (ProgrammaticAdapter / LLMJudgeAdapter / LongitudinalPairsAdapter / L3SchemaManager / cli.py)

代码验证摘要

- 验证源：本项目为当前工作区本地源码 (e:\python_file\agent_practice\poirot) ，等价于 pull_github_repos.py 拉取到本地的源码验证，故不生成拉取 manifest；git commit c1880f8 (2026-08-24, 共 202 个 commit) 。
- 已阅读关键文件：上述 10 项证据对应的 10 个模块文件（类名、方法、公式、阈值已逐行核验）；另阅读 README.md（项目定位）、pyproject.toml（技术栈）、USAGE.md（运行入口与配置）、poirot/backend/agents/leader/（编排入口）、poirot/backend/tests/（测试目录，含 v1/unit/{leader,sandbox,skill,memory} 与记忆检索场景测试 test_retriever.py）。
- 完整性说明：项目共 560+ Python 文件，本次取证聚焦简历最相关的 9 个机制模块；其余模块（TUI、skill 进化、MCP loader 等）未逐行核验，写进简历前如需引用须补验。

简历写法

项目简介

基于 Agent 架构最佳实践从零搭建 Poirot 深度研究 Agent 内核：以 ReAct 循环为骨架，通过上下文预算治理、五层长期记忆、多智能体编排与沙箱隔离构成完整研究 workflow，让主 Agent 在长会话中自主规划检索、调用工具、固化记忆并交付可追溯的结构化研究结论，用于辅助深度信息调研与知识沉淀。

(约 120 字)

负责功能 / 技术难点

- 搭建上下文预算治理链路：按消息 id 去重累计每轮 token 增量，实时计算"当前占用 / 真实模型窗口"占比，分级触发 P1 历史外化、P4 压缩与 P5 工具调用熔断；窗口值不是配置写死，而是穿透 FallbackChatModel 取当前活跃 provider 的真实 context window，多模型切换后熔断阈值依然准确。
- 实现 ReAct 死循环熔断器：在 after_model 钩子对最近 10 条消息做（工具名, 参数哈希）重复检测，同一调用出现 3 次即判定死循环；熔断时保留原消息 id 重建 AIMessage 清空 tool_calls，避免留下无配对 ToolMessage 导致下一轮模型调用 400，同时注入隐藏引导并 jump_to model 强制基于已有信息收尾，避免 token 空耗。
- 设计五层长期记忆中的衰减与持久化：采用 Ebbinghaus 公式在检索时惰性计算记忆强度（强度 = 基础强度 $\times (1 - \text{衰减率})^{\text{小时数} + \text{访问对数加成} + \text{重要度加成}}$ ），不跑后台衰减任务；记忆以单文件 traces.md 为唯一 truth source，内存索引由文件启动加载、增量维护，写操作加线程锁，frontmatter 损坏时记日志跳过而不崩。
- 搭建记忆异步固化管道：daemon 线程 + 队列承接每 N 轮的非阻塞抽取任务，LLM 抽取结果先 JSON 解析容错再编码入库；episodic 记忆总量达到阈值后取最旧的 10 条由 LLM 合并为一条语义记忆；LLM 失败、解析失败、合并失败一律只记日志跳过，绝不影响主 Agent 循环的响应速度。
- 实现多模型故障降级链：按角色优先级组织 provider 链，仅在瞬时故障（超时、连接中断、限流、5xx）时自动降级到下一家，客户端错误（400/401/404）直接抛出避免无效重试；命中后记忆当前活跃 provider，后续轮次直接命中不重复试错，bind_tools 对链上所有模型统一绑定，工具调用能力不因降级而丢失。
- 强制沙箱写路径白名单：对 write_file / str_replace 的写路径与 bash > / >> 重定向目标做挂载区前缀校验（正则捕获绝对路径重定向目标），越界即抛 SandboxPermissionError 让模型改路径重试；同时提供容器路径到宿主机物理路径的反向映射，保证产物写入真实挂载区、artifact 提取链路上拿到的是可操作的 Windows 路径而非容器内路径。

建议简历功能点（完成对应改造后可写）

以下均为需要你本地实际完成改造后才能写进简历的增强项，改造方案见"落地计划"：

- 记忆检索评测闭环：复用现有 L3 评测框架（multiagent/eval 的 EvalBridge 协议 + EvalContext/EvalResult + L3SchemaManager 存储 + Wilson CI），为 HybridRetriever 实现一个 programmatic 评测器（recall@k / MRR 指标，检索样本可扩展 test_retriever.py 的场景数据），量化上下文注入对回答质量的提升。
- 上下文治理可观测面板：把 budget fraction 曲线、P1/P4/P5 触发次数做成结构化事件（现有 RunJournal 事件骨架），能讲清一次长会话中的治理动作时序。
- 工具调用 trace replay：对工具调用链路做回放评测，错误归因到 middleware 层，验证熔断器与守卫在故障注入下的行为。

- 记忆一致性对账任务：定时对账 `traces.md` 与内存索引（当前文件是 `truth source`、索引可重建，补对账任务即闭环）。

可改造方向（2-3 周落地计划）

按"先量化、再固化、最后扩展"排期，每项都产出能讲给面试官的细节：

周次	改造项	具体动作	面试可讲点
第 1 周	记忆检索评测闭环	复用 <code>multiagent/eval</code> 现有框架（ <code>EvalBridge</code> + <code>EvalContext/EvalResult</code> + <code>L3SchemaManager</code> 存储 + <code>Wilson CI</code> ），为 <code>HybridRetriever</code> 实现 <code>programmatic</code> 评测器（ <code>recall@k / MRR</code> ），样本基于 <code>test_retriever.py</code> 场景数据扩展，跑通 <code>cli.py</code> 评测入口	评测器怎么挂进 <code>EvalBridge</code> 、 <code>BM25</code> 参数怎么调、 <code>recall@k</code> 基线多少、为什么用惰性衰减不用后台任务
第 2 周	治理可观测性 + 一致性对账	为 <code>budget / 熔断 / 记忆事件</code> 补结构化日志与触发计数，实现 <code>traces.md</code> 与内存索引对账任务	一次真实长会话中 <code>P1→P4→P5</code> 的触发时序、对账怎么发现并修复索引漂移
第 3 周	工具链路故障注入回放	复用 <code>LoopDetection</code> 与沙箱守卫，写故障注入测试（工具抛错、路径越界、模型限流）验证熔断与降级行为	死循环熔断的判定与消息修复细节、降级链在故障下的行为与取舍

落地风险控制：本项目机制密度高，2-3 周只够完成上述 3 项；不要贪多把未完成的改造写进简历。每完成一项，用一条"业务问题 → 方案 → 失败处理 → 结果"的句式更新简历，并在 `backend/tests/` 中补对应测试。

最终建议

- 简历只需保留"项目简介 + 6 条负责功能"，每条都能被追问到类名、阈值、公式或失败处理——这是本项目相对普通"接 API 项目"的核心优势。
- 面试自检：任选一条，你能否讲清它的数据流（消息从哪来、状态存哪、失败怎么办）？讲不清的条目在写进简历前先回源码确认。
- 不建议把"建议简历功能点"当作已有经历包装；第 1-3 周改造完成后，它们才转为"负责功能"。